

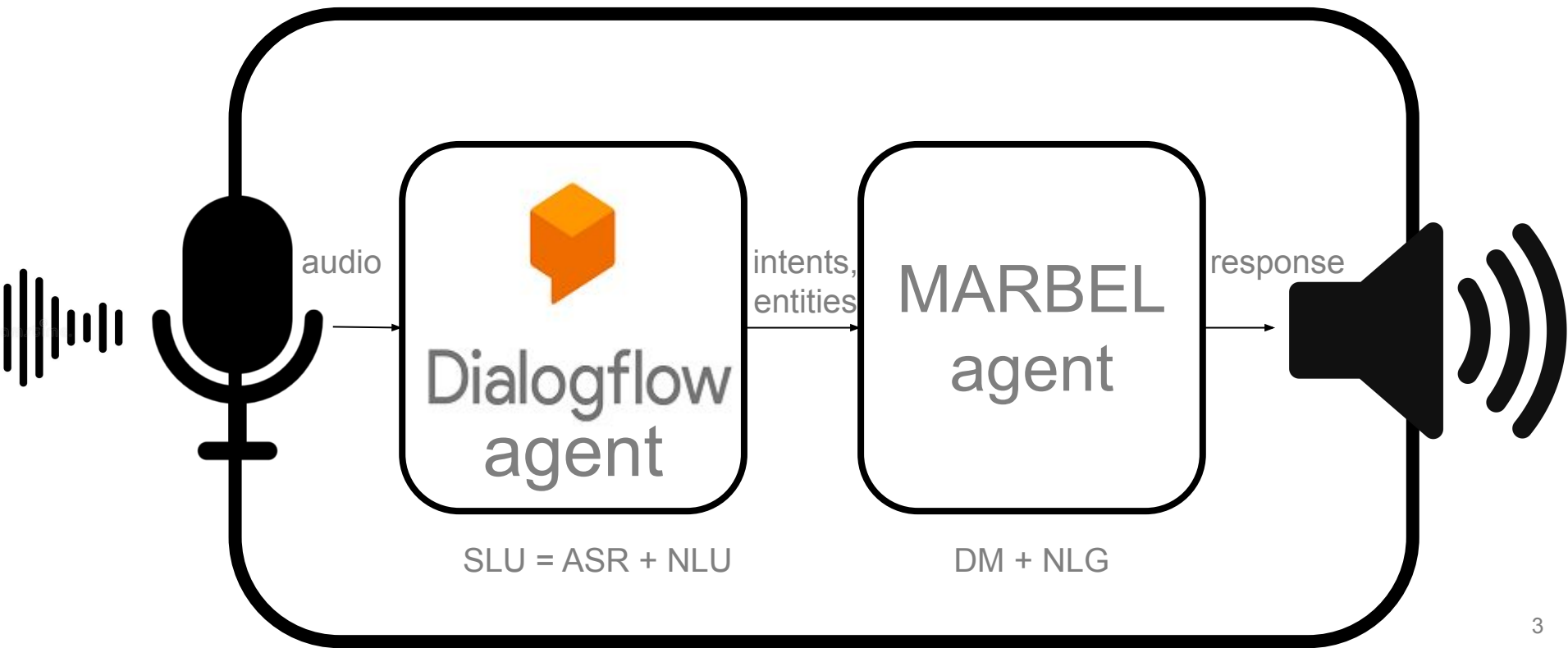
Speech Language Understanding Pipeline (ASR + NLU)

Project Conversational Agents - Lecture 2

Overview

- SLU and its Connection
- Language Modeling with Transformers
- Introduction of ASR with WHISPER
- Your NLU Model with BERT
- Demo & Test Cases
- Summary

Our previous conversational agent



Challenge, Opportunity, Objective

Challenge:

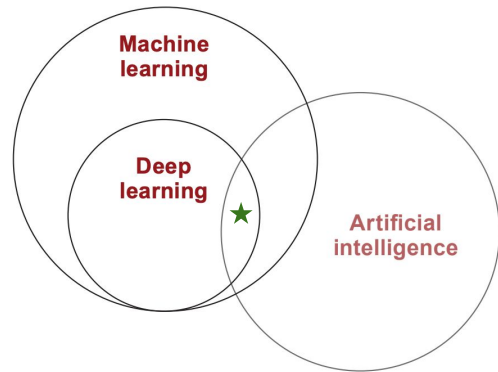
- Dialogflow is a black box with limited choice of machine learning models.
- Pre-built models may not be accurate for specific domains, requiring extra tuning or training to improve precision.

Opportunity:

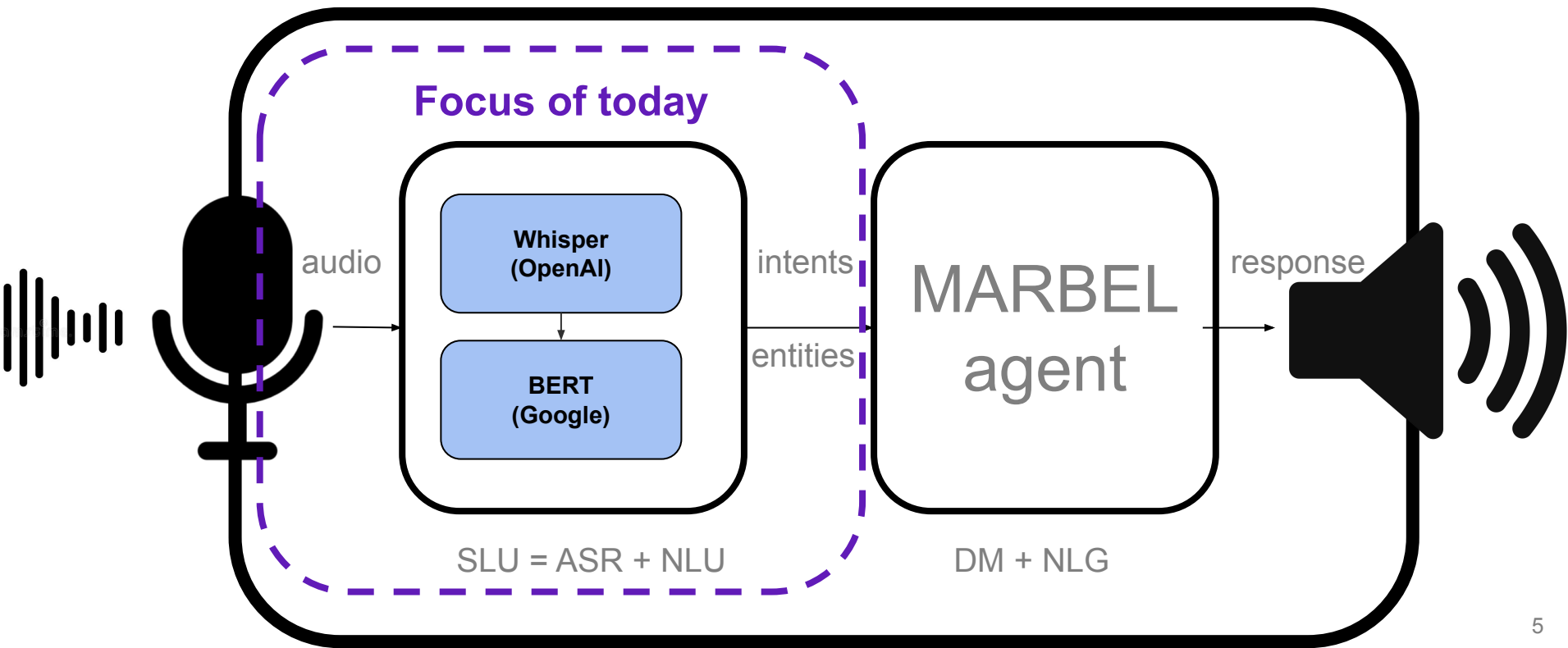
- A greater control and allows for continuous improvement.
- An accessible way to explore and apply modern AI models.

Objective:

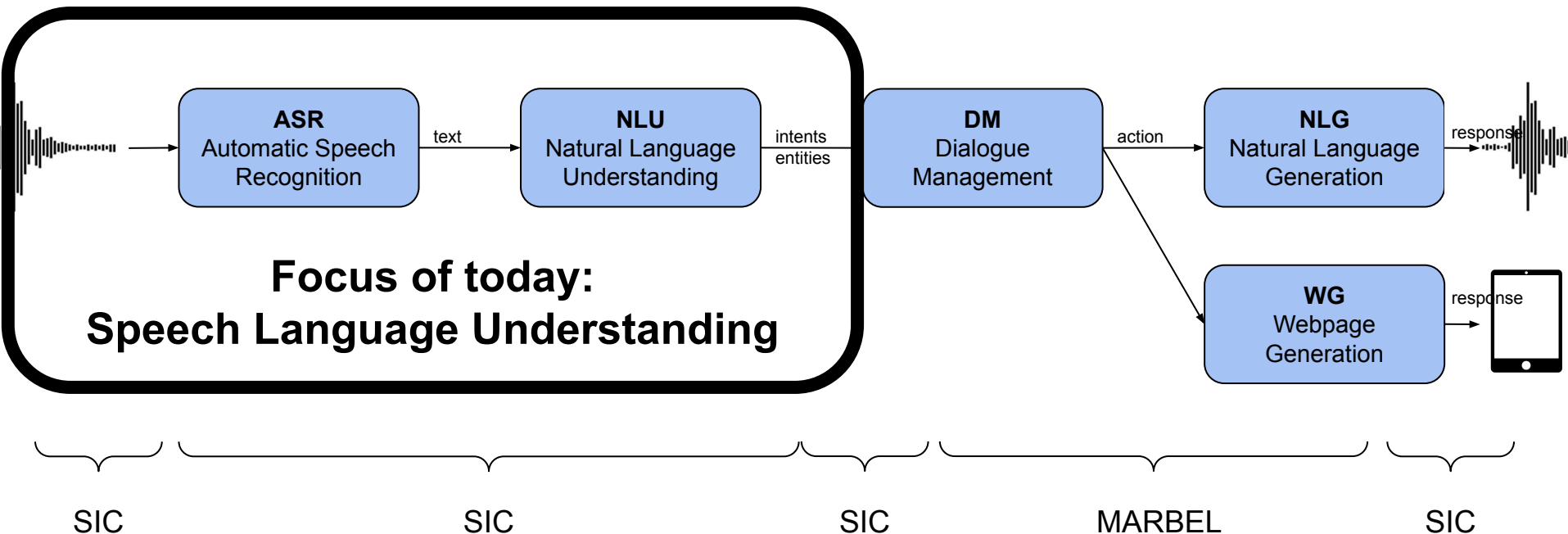
- ★ Hands-on a simple NLU model with basic understanding of theory.



Agent with our machine learning models



Pipeline Architecture: In this project



ASR + NLU - Flow



ASR + NLU - Flow

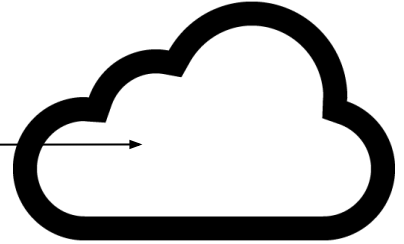
NLU: BERT



Intent: greeting (100%)
Entities: -



SIC & EIS (connector)

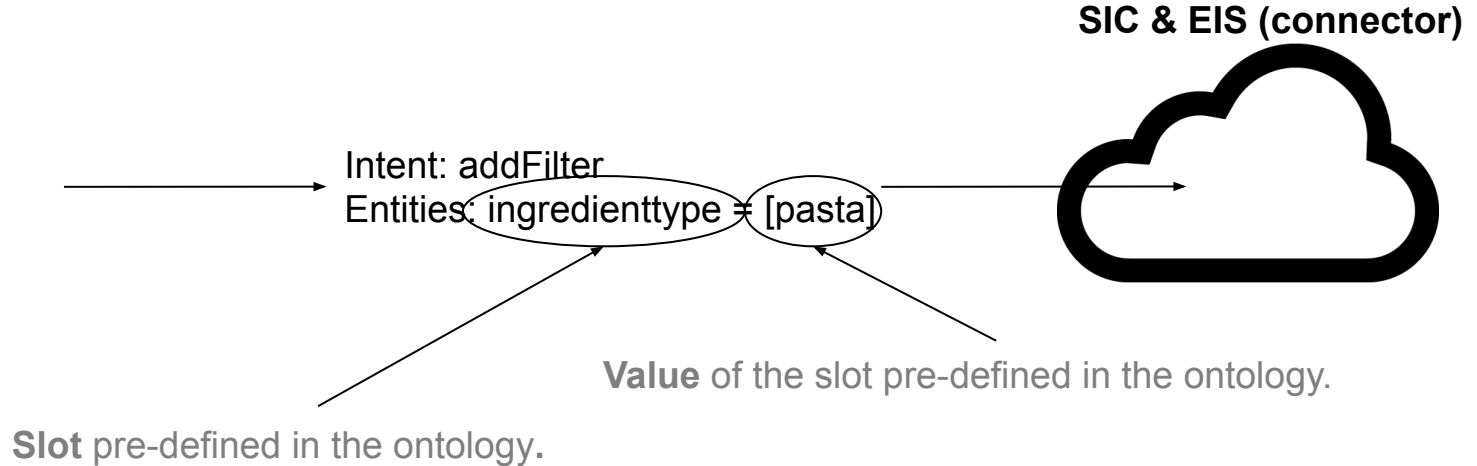


ASR + NLU - Flow



ASR + NLU - Flow

NLU: BERT



Overview

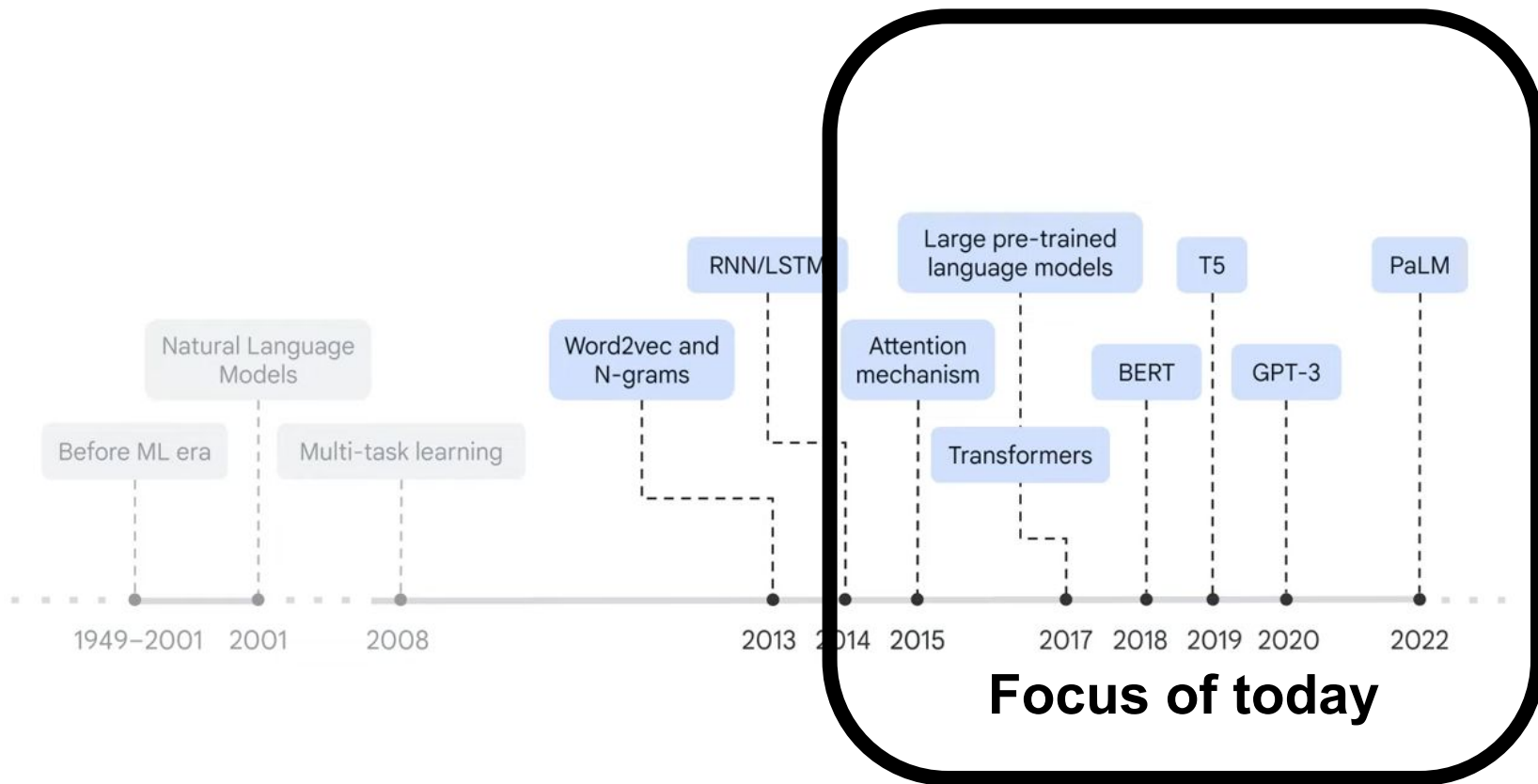
- SLU and its Connection
- Language Modeling with Transformers
- Introduction of ASR with WHISPER
- Your NLU Model with BERT
- Demo & Test Cases
- Summary

What is a language model?

- Probability distribution over a sequence of words.
 - Given a sequence of words as context (**observation**) in a particular "language," it predicts which word (**output**) is most likely to follow.
- An example
 - $p1 = P(\text{"a quick brown fox"})$
 - $p2 = P(\text{"fox quick a brown"})$
 - $p3 = P(\text{"un chien quick brown"})$
 - $p4 = P(\text{"un chien brun rapide"})$
 - NLP: $p1 > p2 > p3 > p4$



Language modeling history



Pretrained transformer models

Categories

Encoder & Decoder



Bidirectional and Auto-Regressive Transformer

Decoder only

GPT-3

GPT-2

Generative Pretrained Transformer

Encoder only



Bidirectional Encoder Representations from Transformers

Models



OpenAI Whisper

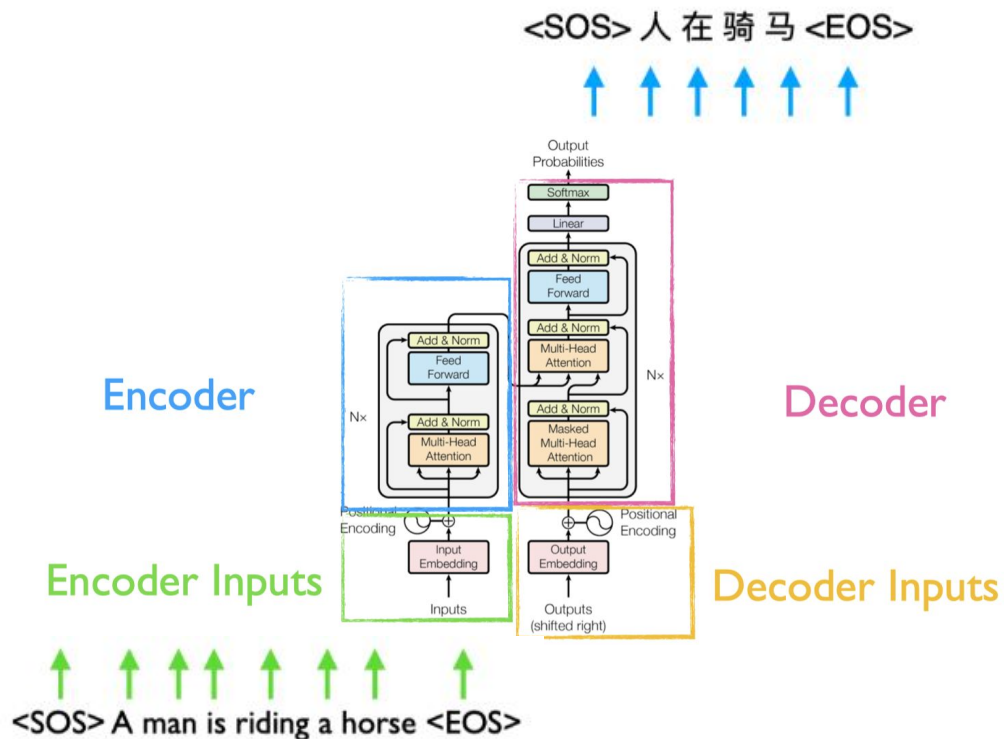


OpenAI
ChatGPT



BERT

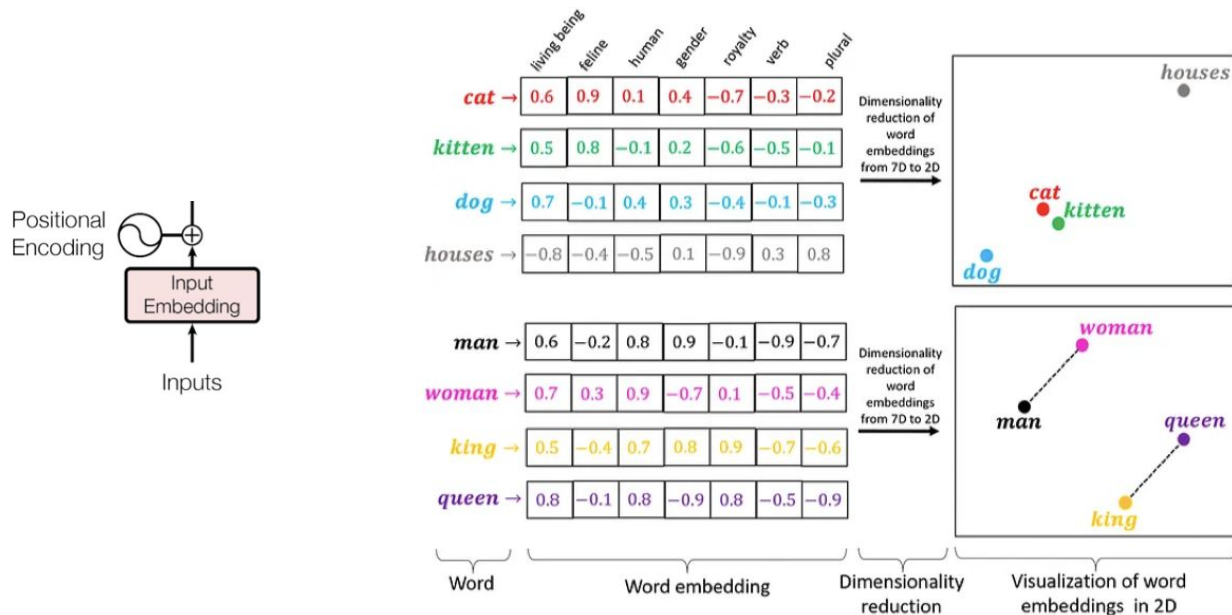
Transformers: Architecture



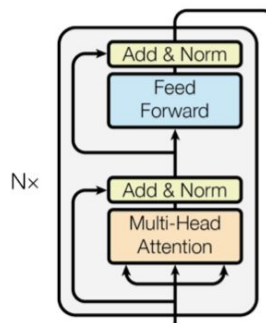
- **Encoder**
builds deep representations of the input.
- **Decoder**
uses these representations to produce coherent, context-aware output, while managing the sequential nature of generation.

Transformers: Input Embedding

- **Tokenization**: breaking down text into smaller units, typically words or phrases, called tokens.
- **Word embedding**: converting words into numerical representations (vectors), called word embeddings.



Transformers: Encoder



N times of encoder blocks

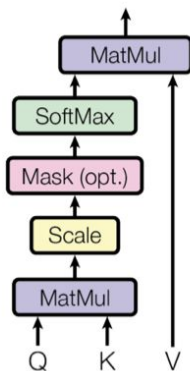
- **Multi-Head Attention**
Focuses on capturing relationships within the input sequence.
- **Add**
Allows both original and residual information flow in the networks.
- **Norm**
Conducts layer normalization to stable network and reduce training time.
- **Feed forward:**
Projects the attention output potentially give it richer representation.

Transformers: Self-Attention

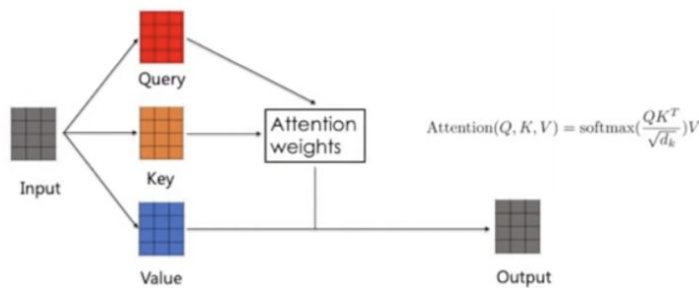
Aim to help the models associate each word in the input with other words.

- **Query** – the current token that is trying to gather information from other tokens in the sequence.
- **Key** – the other tokens in the sequence that provide relevant information to the current token.
- **Value** – the actual information from the tokens, which will be aggregated based on how relevant they are to the query.
- **Output**: a weighted sum of values. The weights are determined by the dot-product of the query with all the keys.

Scaled Dot-Product Attention

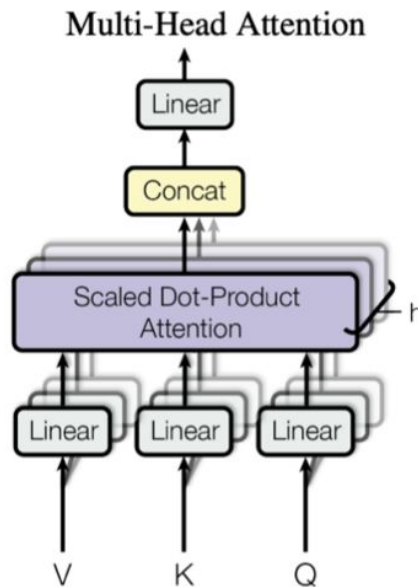


Query, Key, and Value



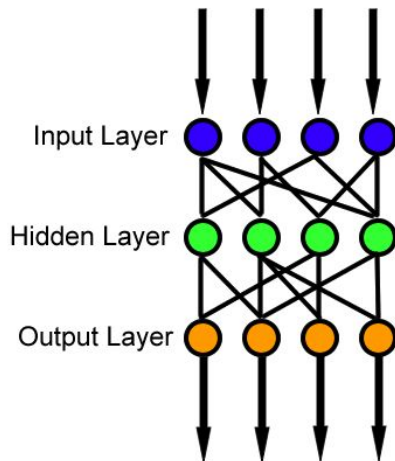
Transformers: Multi-Head Attention

- Multiple self-attention mechanism in parallel (i.e., heads).
- Allows the model to attend different aspects of input sequence simultaneously.



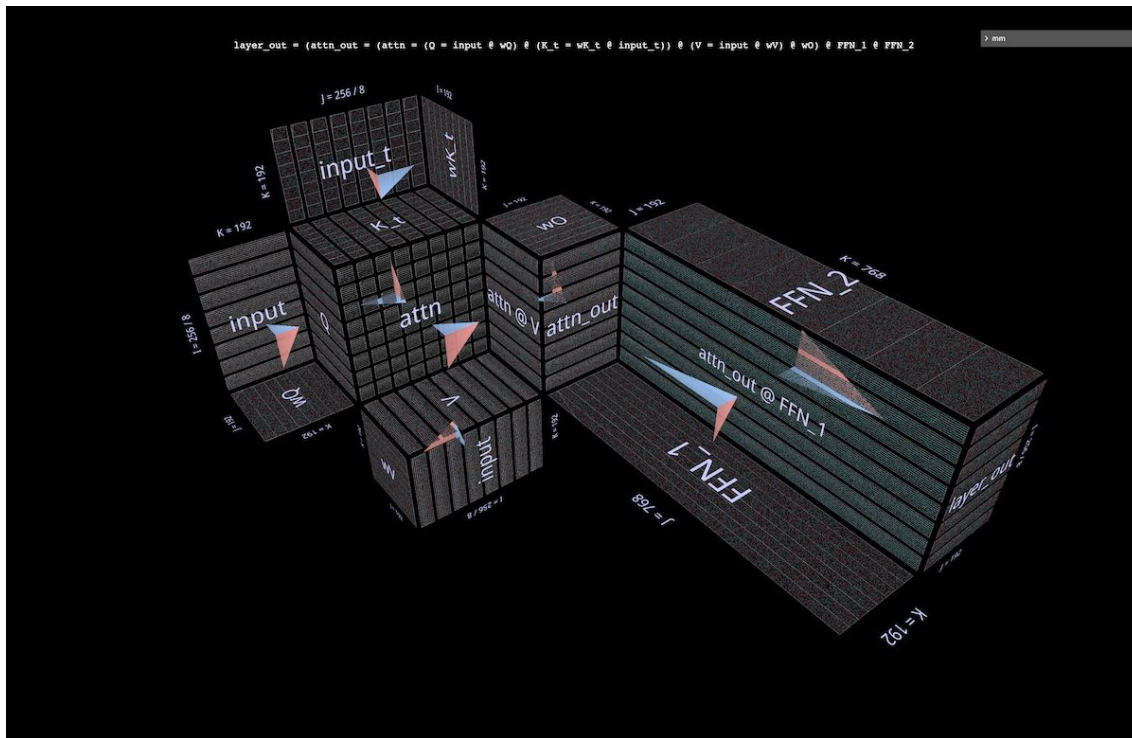
Feed Forward

- Passing information forward through the network, from the input layer, through the hidden layers, and finally to the output layer.
- Analogy: Imagine you're at a party, and you're trying to pass a message to a friend on the other side of the room.



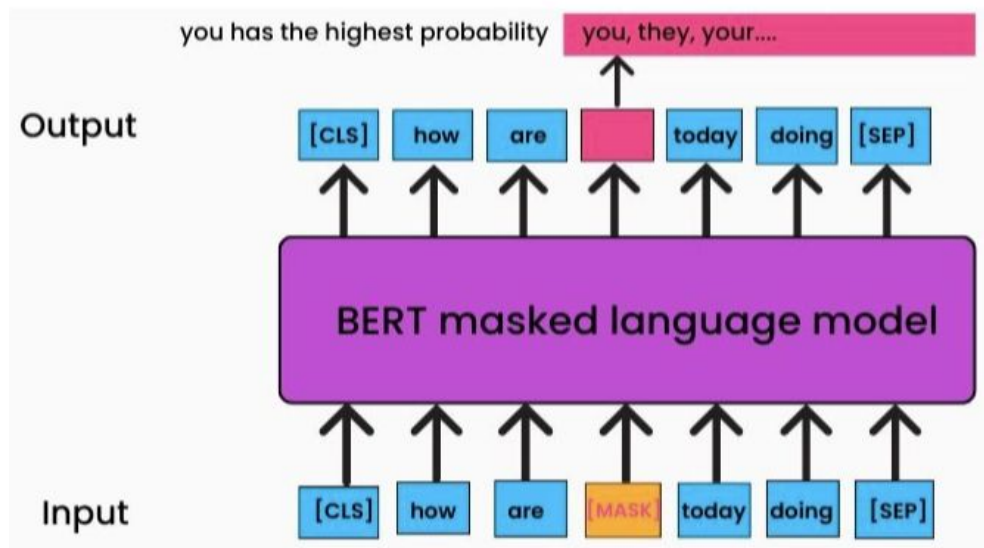
Multi-layer perceptron (MLP) with at least three layers of nodes: an input layer, an output layer, and one or more hidden layers.

Transformers are a set of computational matrices



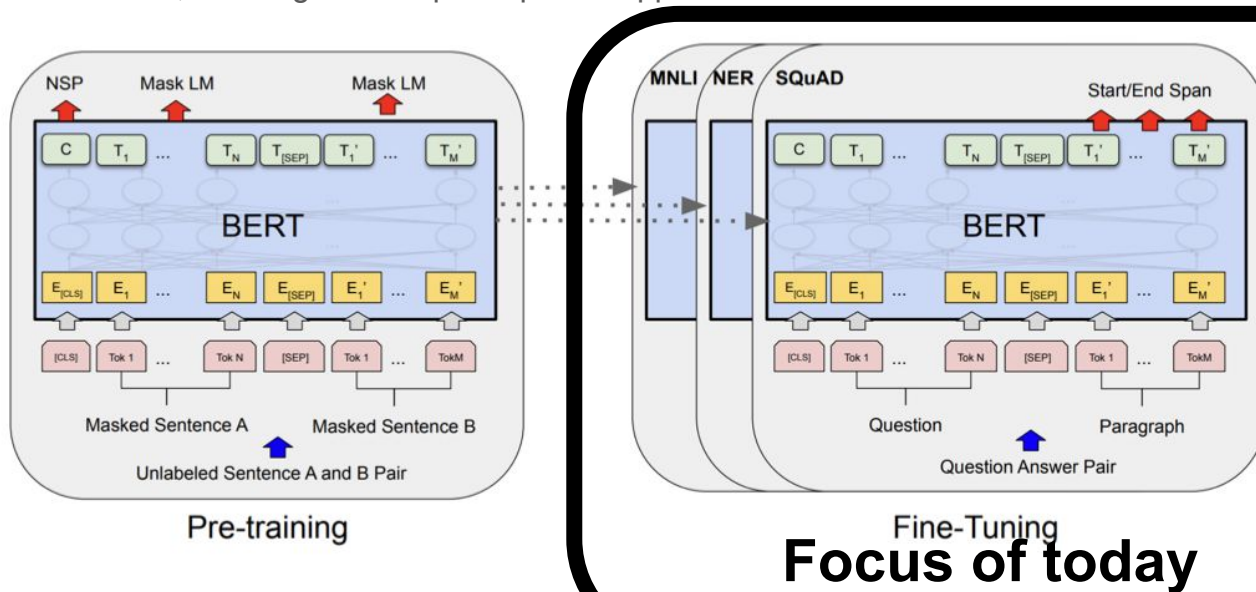
BERT: Bidirectional Transformer Encoder

- BERT: Bidirectional Encoder Representations from Transformers
- Context-aware word representations for various natural language understanding tasks
- Process all the entire input sequence at once.



BERT: Pretraining and Fine-tuning

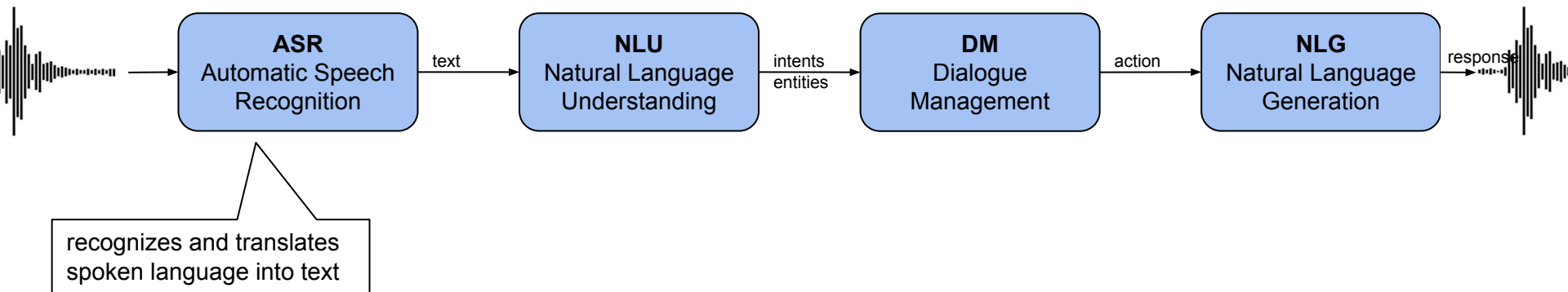
- **Pretraining:** trained on a large corpus of text in a *self-supervised* manner. This means it is trained on large-scale, raw, unlabeled text without human annotations, using an automatic process to generate input-output pairs from the text.
- **Fine-tuning:** first initialized with its pretrained parameters, and then all parameters are fine-tuned using labeled data from downstream tasks, allowing it to adapt to specific applications.



Overview

- SLU and its Connection
- Language Modeling with Transformers
- Introduction of ASR with WHISPER
- Your NLU Model with BERT
- Demo & Test Cases
- Summary

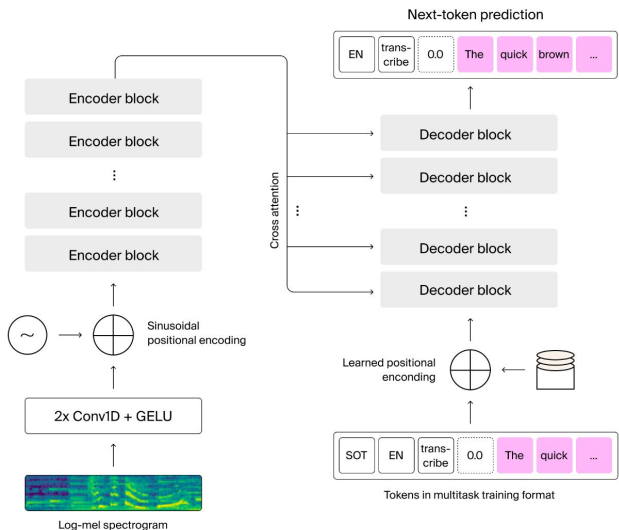
Pipeline Architecture



ASR with WHISPER

- Automatic speech recognition (ASR), convert speech into text (transcript)
- OpenAI Whisper, one of the most comprehensive ASR systems available.
 - An encoder-decoder Transformer
 - Trained on a vast dataset of 680,000 hours of supervised data

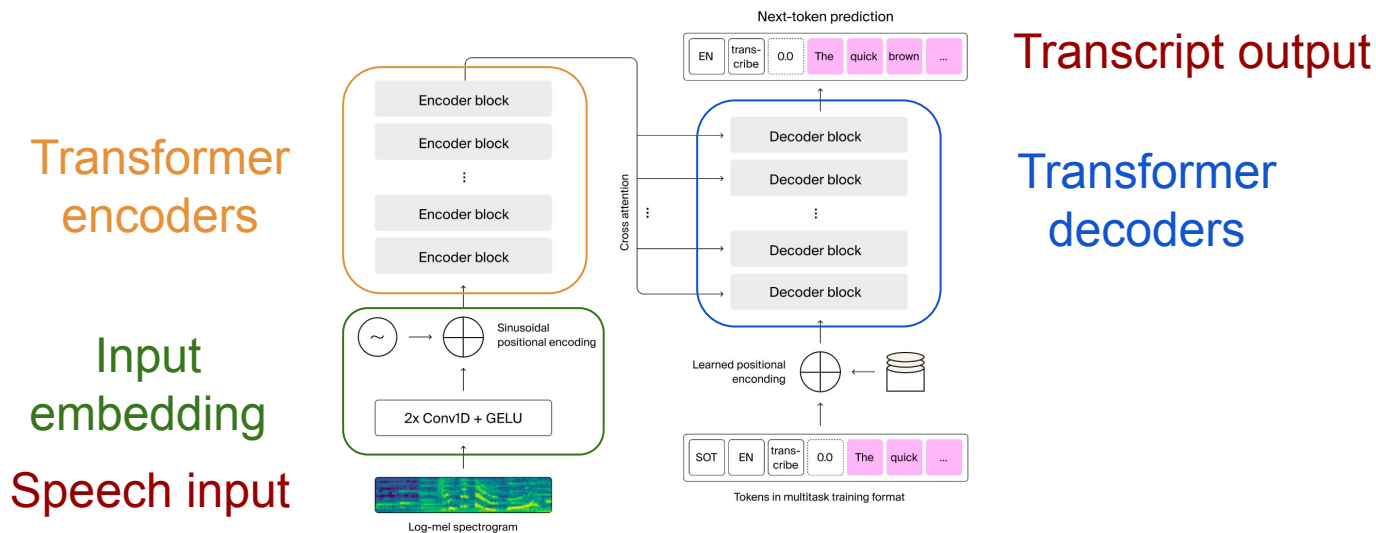
Speech input



Transcript output

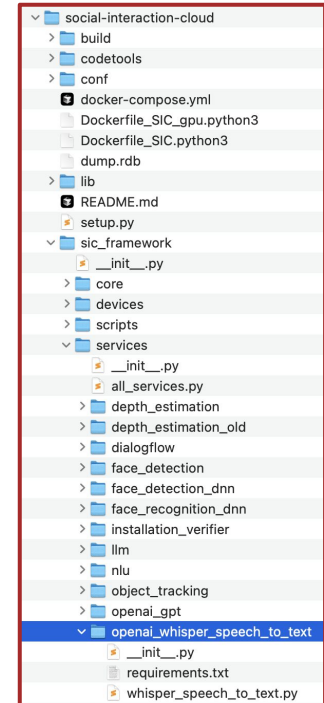
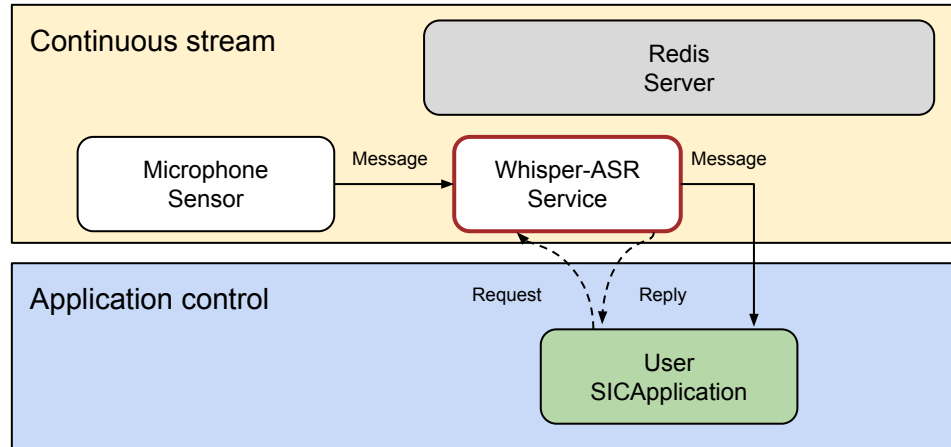
ASR with WHISPER

- Automatic speech recognition (ASR), convert speech into text (transcript)
- OpenAI Whisper, one of the most comprehensive ASR systems available.
 - An encoder-decoder Transformer
 - Trained on a vast dataset of 680,000 hours of supervised data



Whisper on the Social Interaction Cloud (SIC)

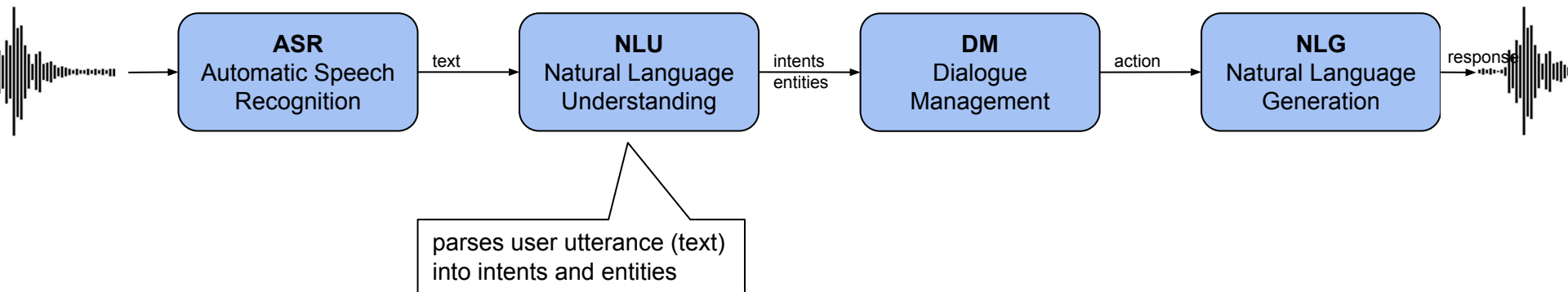
- OpenAI Whisper is available on SIC-v2
 - Capable of running both local (for free) and in the cloud.
 - You can directly use in your SICApplication (ASR+NLU pipeline)



Overview

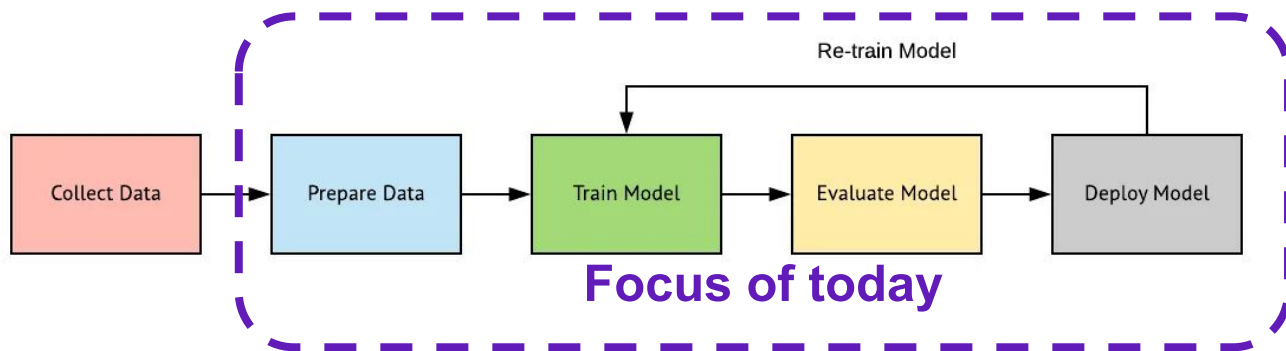
- SLU and its Connection
- Language Modeling with Transformers
- Introduction of ASR with WHISPER
- **Your NLU Model with BERT**
- Demo & Test Cases
- Summary

Pipeline Architecture



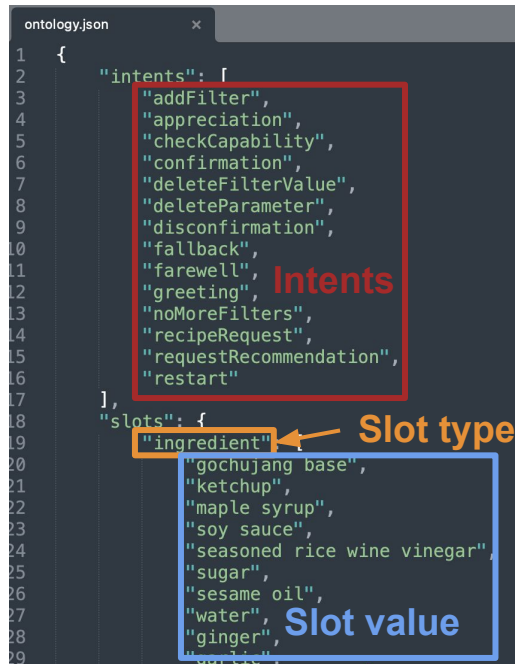
Understand your task

- **What you are building:** A model that uses BERT, a pre-trained language model, to classify user intents (e.g., *"find_recipe"*) and identify specific slots (e.g., *"cuisine: Japanese"*).
- **Key Components:**
 - **Intent Classification:** Determines the overall goal of the user.
 - **Slot Filling:** Extracts detailed information from the input text.
- The machine learning process



Ontology lists all intents and slot-values

- The ontology is a dictionary where all the possible intents and slots for the system are defined.
 - **Intents** represent the user's high-level intentions or goals.
 - Example: When a user says, "Can you recommend a recipe?", the intent could be `requestRecommendation`.
 - **Slots** define specific pieces of information extracted from the user's input.
 - Example: In the query "Add garlic and chicken thighs to the recipe filter," garlic and chicken thighs are slot values of the `ingredient` slot type.



The image shows a code editor window titled 'ontology.json'. The JSON content is as follows:

```
1 {
2   "intents": [
3     "addFilter",
4     "appreciation",
5     "checkCapability",
6     "confirmation",
7     "deleteFilterValue",
8     "deleteParameter",
9     "disconfirmation",
10    "fallback",
11    "farewell",
12    "greeting",
13    "noMoreFilters",
14    "recipeRequest",
15    "requestRecommendation",
16    "restart"
17  ],
18  "slots": {
19    "ingredient": [
20      "gochujang base",
21      "ketchup",
22      "maple syrup",
23      "soy sauce",
24      "seasoned rice wine vinegar",
25      "sugar",
26      "sesame oil",
27      "water",
28      "ginger",
29      "garlic"
30    ]
31  }
32 }
```

Annotations in the image:

- A red box highlights the `"intents": [ ... ]` array. A red arrow points to it from the word **Intents** written in red.
- An orange box highlights the `"ingredient": [ ... ]` array. An orange arrow points to it from the text **Slot type** written in orange.
- A blue box highlights the list of ingredients inside the `"ingredient"` array. A blue arrow points to it from the text **Slot value** written in blue.

We provide `ontology.json` file by summarizing all the intents, slots and their values in the training dataset.

Synonyms for slot normalization

- We define a synonym dictionary where all the possible synonyms of a value for each slot (provided in synonyms.json).
- We provide the helper functions (in utils.py) for loading and normalizing the synonyms.

**Canonical slot
value in the
ontology**

```
synonyms.json
},
"dietaryRestriction": {
  "vegetarian": ["vegetarian", "veggie", "vegetarians", "Vegetarian", "vegatarian"],
  "vegan": ["vegan", "plant-based", "vegans", "Vegan", "Plant-based", "non-vegan"],
  "lactose-free": ["lactose-free", "lactose free", "dairy free", "diary-free", "dia"],
  "pescatarian": ["pescatarian", "pescatarians", "pescetarian", "Pescatarian", "pes"],
  "gluten-free": ["gluten-free", "Gluten-free", "gluten free", "without gluten", "f"],
  "spicy": ["spicy", "Spicy", "spice", "heat", "non-spicy", "non spicy"],
  "nut-free": ["nut-free", "nut free", "without nuts", "can't eat nuts", "allergy t"],
  "halal": ["halal"],
  "keto": ["keto", "low-carb", "low-carb recipes"],
  "paleo": ["paleo", "Paleo"],
  "healthy": ["healthy", "low-fat", "high-protein", "high-protei"],
  "soy-free": ["soy-free"],
}
```

List of synonyms

Prepare the dataset for model inputs

- **Why?** Models can only process numerical data or matrices (tensors) as input.
- **Before:**
 - Raw json data includes plain text, an intent label, and slots with values.

```
{
  "id": "st041",
  "text": "I'd like a meal that's fast to prepare.",
  "intent": "addFilter",
  "slots": {
    "duration": "fast"
  }
}
```

- **After:**
 - Tokenized, encoded, padded, structured data ready for model input.

```
{
  'input_ids': torch.tensor([101, 1045, 1521, 1005, 1040, 2066, 1037, 7953, 2008, 15]),
  'attention_mask': torch.tensor([1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0]),
  'intent_label': torch.tensor(0), # Encoded "addFilter"
  'slot_labels': torch.tensor([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0]) # BIOES tag
}
```

Prepare the dataset: Tokenization

- The text is broken into smaller units (tokens) using the BERT tokenizer.

Output

```
['i', '', 'd', 'like', 'a', 'meal', 'that', '', 's', 'fast', 'to', 'prepare', '.']
```

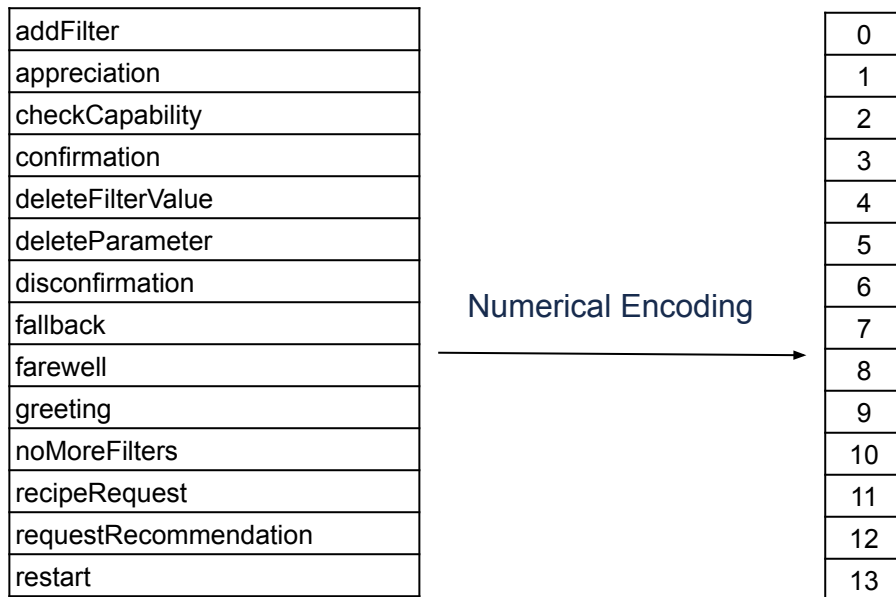
BERT tokenizer

Input

```
I'd like a meal that's fast to prepare.
```

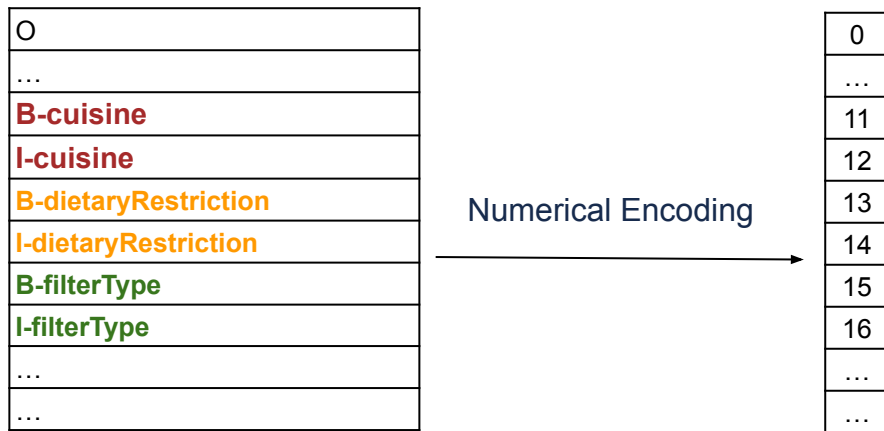
Prepare the dataset: Intent Label Encoding

- The intent is mapped to a numerical label using the intent encoder.



Prepare the dataset: Slot Label Encoding

- First, slot labels are converted into **BIO** format:
 - B-**{slot}**: **B**eginning of a slot entity.
 - I-**{slot}**: **I**nside a slot entity.
 - O: **O**utside of any slot entity.
- Then, all slot labels mapped to numerical labels using the slot encoder.



```
['O'] + [f'B-{{slot}}' for slot in slots.keys()]  
      + [f'I-{{slot}}' for slot in slots.keys()]
```

Why BIO Format?

- This labeling scheme helps identify the boundaries of multi-token slot entities, for example,

Plain text	How	to	cook	an	Italian	pasta
BIO-format slot tags	O	O	O	O	B-cuisine	I-cuisine

Prepare the dataset: Padding and Truncation

- Sequences are padded or truncated to a fixed length to fit with numerical tensors.
 - Tokens are padded with special token [PAD] and its ID is set as zero.
 - BIO tags are padded with O.
 - If we fix the max length as 8 tokens, here are two examples:

Tokens	How	to	cook	an	Italian	pasta	?	[PAD]
Tags	O	O	O	O	B-cuisine	I-cuisine	O	O

Tokens	i	,	d	like	a	meal	that	,										
Tags	O	O	O	O	O	O	O	O										

← max length as 8 tokens → **Truncated after 8!**

Hands-on understanding of your data via the guide! Think about:

Dataset Balance:

- Are the distributions skewed?
- How might this affect the model?
- Are all intents and slots well-represented, or are some missing?

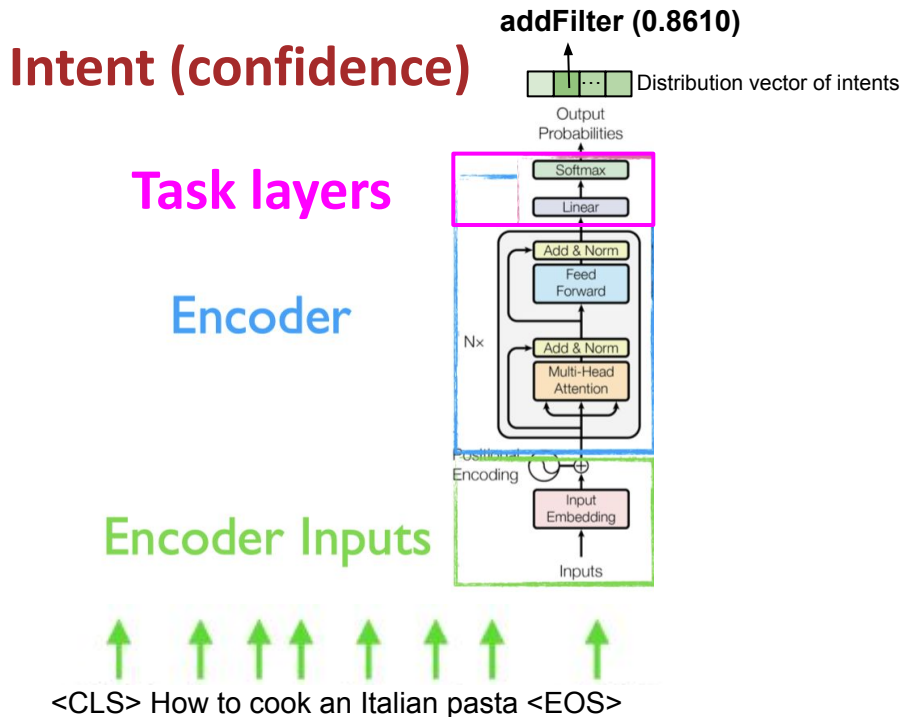
Test Dataset Representativeness:

- Does the test set reflect the training set's distribution?
- If not, how might this impact evaluation?



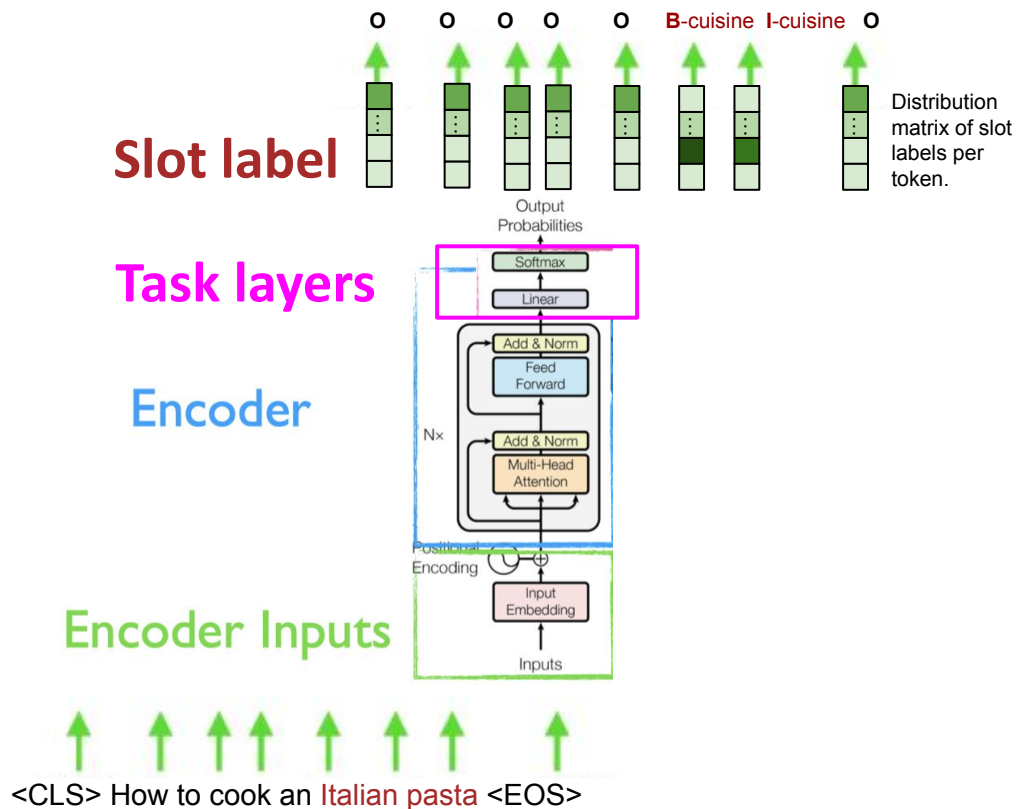
NLU: Intent classifier

- **Intent Classifier:**
 - **What it does:** Classifies the entire sentence's intent.
 - **Input size:** The size of the pooled output from BERT. (**Hint:** This would be the `hidden_size` of the last BERT layer)
 - **Output size:** The number of intents in your dataset.
- BERT provides a special [CLS] token embedding for the entire input text, which is used to classify intents by passing it through a linear layer that maps to the number of intent classes.

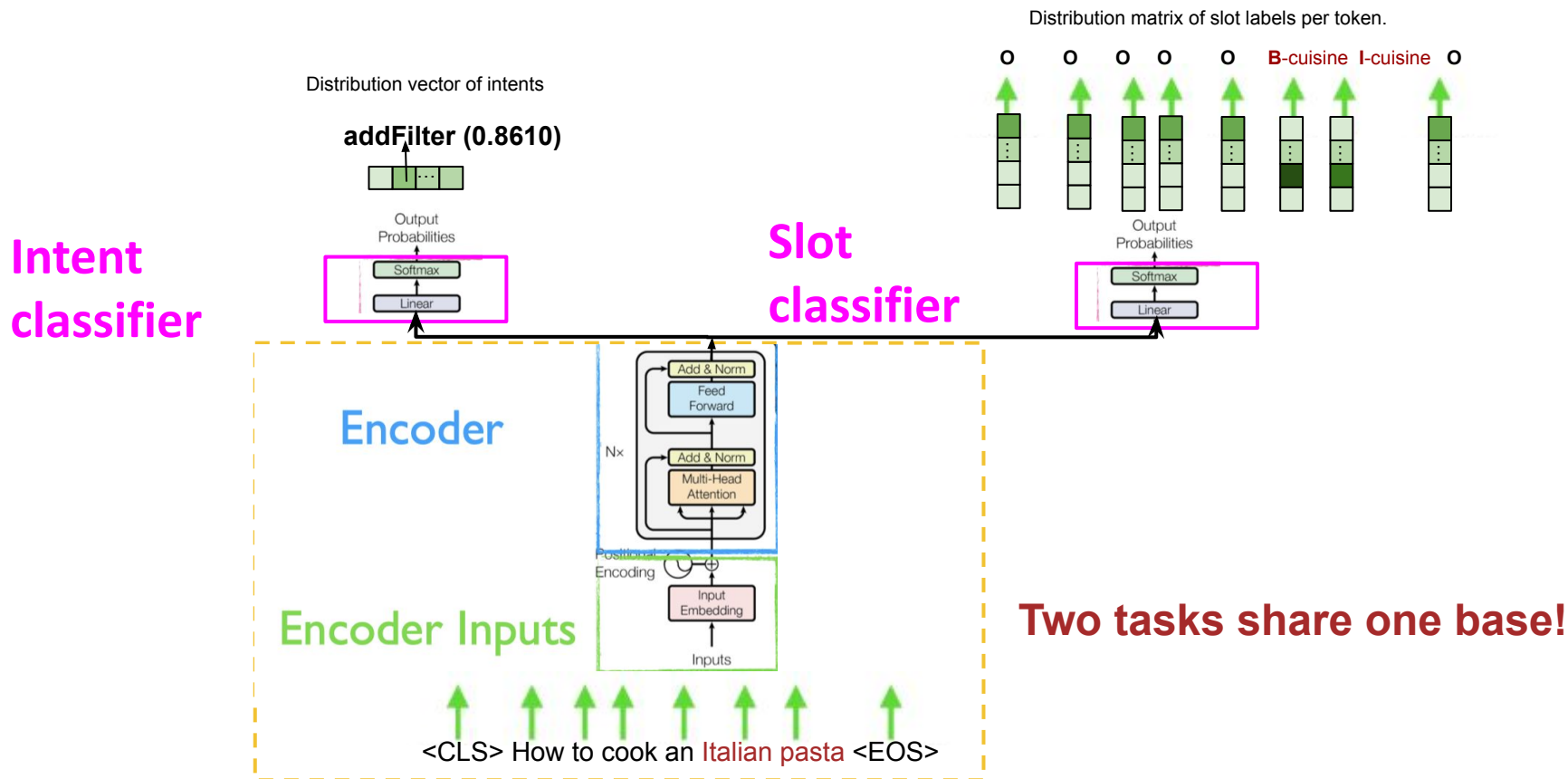


NLU: Slot classifier

- **Slot Classifier:**
 - **What it does:** Predicts a slot label for each token in the input.
 - **Input size:** The size of the last hidden state embeddings from BERT.
 - **Output size:** The number of slot labels in your dataset.
 - Converts slot types and their corresponding **BIO-format tags** (B-slot, I-slot, O) to get all your slot labels
- Use the last hidden state for each token in each sequence of the batch to classify slots by passing them through another linear layer.



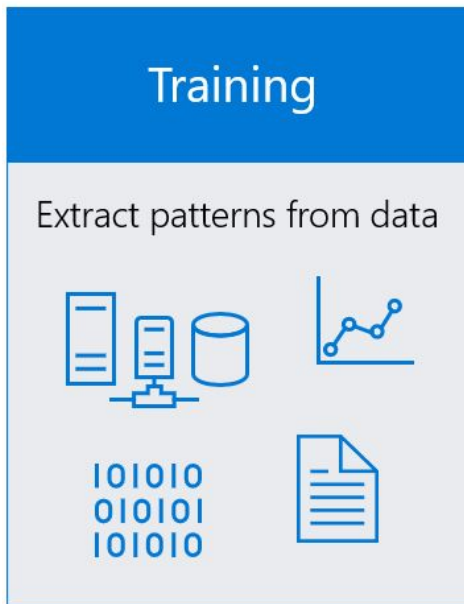
NLU model: Architecture with Two Classifiers



Training Your Model

- **What is Training?**

- Extract pattern from data
- Optimize a model to make better predictions (intents and slots) by minimizing a loss function.

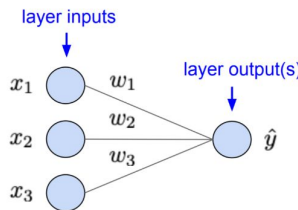


Source: <https://learn.microsoft.com/tr-tr/windows/ai/windows-ml/what-is-a-machine-learning-model>

Training Your Model

- What is Training?

- In a neural network,
 - Learn the parameters W in the model from given the dataset $D=\{(X, Y)\}$, where the vector $X=[x_1, x_2, x_3]$.
 - Optimize by minimizing a loss function defined by predicted labels $\hat{y}$ and ground true labels y .
- In one Loop
 - **Forward Pass:** The input data flows through the model to generate predictions.
 - **Loss Calculation:** The predictions are compared with ground truth labels to calculate the loss.
 - **Backward Pass (Backpropagation):** The model adjusts its weights based on the loss to improve predictions in the next iteration.



Output: $\hat{y} = w_1x_1 + w_2x_2 + w_3x_3 + b$
 $= w^T x + b$

Neural network parameters:
 $W = \{[w_1, w_2, w_3], b\}$

Often refer to all parameters together as just "weights". Bias is implicitly assumed.

bias term (allows constant shift)

layer "weights" layer bias

Assignment: Completing the Training Function

```
48 """
49 Usage:
50 - Import 'train_model' and call it with the preprocessed dataset and intent_slot_classification_model configuration:
51 from train import train_model
52 intent_slot_classification_model = train_model(...)
53 """
54
55 import torch
56 from torch.utils.data import DataLoader
57 import torch.nn as nn
58 from sic_framework.services.nlu.utils.dataset import NLURecipeDataset
59 from sic_framework.services.nlu.utils.model import BERTNLModel
60
61
62 def train_model(model, dataset, num_epochs=3, batch_size=2, learning_rate=5e-5, device="cpu"):
63     """
64     Trains a BERT-based Natural Language Understanding (NLU) model for intent classification and slot filling.
65
66     Args:
67         model (BERTNLModel): The BERT-based model to be trained.
68         dataset (NLURecipeDataset): A preprocessed dataset ready for training.
69         num_epochs (int): The number of epochs to train the model. Default is 3.
70         batch_size (int): The size of data batches used in training. Default is 2.
71         learning_rate (float): The learning rate for the Adam optimizer. Default is 5e-5.
72         device (str or torch.device): The device to use for training ("cpu" or "cuda"). Default is "cpu".
73
74     Returns:
75         BERTNLModel: The trained model, ready for evaluation or inference.
76     """
77     # Move the model to the specified device (e.g., CPU or CUDA)
78     # model.to(device)
79
80     # Define the loss functions for intent and slot classification
81     # intent_loss_fn -> Use nn.
82     # slot_loss_fn -> Use nn.
83
84     # Define the optimizer (use torch.optim.Adam with the model parameters and learning_rate)
85     # optimizer -> Initialize with model.parameters() and lr=learning_rate
86
87     # Prepare the DataLoader
88     # dataloader -> Use DataLoader(dataset, batch_size=batch_size, shuffle=True)
89
90     # Training loop for the specified number of epochs
91     for epoch in range(num_epochs):
92         # Set the model to training mode
93         # model.train()
94
95         # For batch in dataloader:
96             # Zero out gradients from the previous step
97             # optimizer.zero_grad()
98
99             # Move inputs and labels to the specified device
100             # input_ids -> Extract from batch['input_ids'] and move to device
101             # attention_mask -> Extract from batch['attention_mask'] and move to device
102             # intent_labels -> Extract from batch['intent_label'] and move to device
103             # slot_labels -> Extract from batch['slot_labels'] and move to device
104
105             # Perform a forward pass through the model
106             # intent_logits, slot_logits -> Call the model with input_ids and attention_mask
107
108             # Compute the intent and slot losses
109             # intent_loss -> Compute using intent_loss_fn and intent_logits, intent_labels
110             # slot_loss -> Compute using slot_loss_fn, slot_logits.view(-1), and slot_labels.view(-1)
111
112             # Combine the intent and slot losses
113             # loss -> intent_loss + slot_loss
114
115             # Backpropagation step - send loss backward
116             # loss.backward()
117
118             # Update model parameters with optimizer
119             # optimizer.step()
120
121             # Print progress for the current epoch and batch loss
122             # print(f'Epoch {epoch + 1}, Loss: {loss.item()}')
123
124     # Return the trained model
125     # return model
```

 There are function(s) to be completed in this section!

 Due to how the imports are set up and because the intent and slot classifier is part of the social-interaction-cloud python package when you make changes you need to make sure to reinstall the social interaction cloud via `pip install .`

Develop Your NLU Model at

/pca-agent-2026/social-interaction-cloud/sic_framework/services/nlu/utils

```
1 The repository is structured as follows:
2 utils/
3 |─ checkpoints/      # Directory for saving trained model checkpoints
4 |─ data/             # Directory containing data files (ontology, train/test datasets)
5 |   │─ ontology.json # Ontology file containing intents, slots, and synonyms
6 |   │─ train.json    # Training dataset
7 |   │─ test.json     # Test dataset
8 |   └─ synonyms.json # Synonyms for slot normalization
9 └─ data_processing.py # Utilities for additional data preprocessing (if needed)
10 └─ dataset.py        # Dataset preparation and preprocessing module
11 └─ evaluation.py     # Model evaluation and metrics generation
12 └─ run_train_test.py # Main script to run training, evaluation, and inference
13 └─ model.py          # Defines the BERT-based model architecture
14 └─ predict.py        # Inference module for predicting intents and slots
15 └─ requirements.txt   # Python dependencies for the project
16 └─ train.py          # Training module for the intent-slot classifier
17 └─ utils.py          # Helper functions for argument parsing, slot normalization, and synonym resolution
18
```

 Due to how the imports are set up and because the intent and slot classifier is part of the social-interaction-cloud python package when you make changes you need to make sure to reinstall the social interaction cloud via `pip install .`

Evaluating Your Model

- **What is Evaluating?**
 - Understand how well the trained model performs on the test dataset.
 - Identify weaknesses to figure out where to improve.
 - **Metrics** (Criteria)
 - **Accuracy**: percentage of correct intent (slot) label predictions made by the model.
 - **Precision**: How many predicted intents (slots) are correct.
 - **Recall**: How many true intents (slots) are correctly identified.
 - **F1-Score**: The harmonic mean of precision and recall, offering a balanced measure of accuracy for each intent (slot).

Inference with Your New Data

Output

```
addFilter 0.982801616191864 {'cuisine': 'japanese', 'mealType': 'lunch.'}
```

↑ Inference (prediction)

BERT-NLU
model

Input

```
Find me a Japanese recipe for lunch.
```

Improve and Retrain Your Model

Hyperparameter Tuning

1. Learning Rate
2. Number of Epochs
3. Batch Size
4. Optimizer
5. Dropout Rate
6. Maximum Sequence Length

Architecture

Architecture Improvements

1. Add Dropout for Regularization
2. Add a Dense Layer for Task-Specific Features
3. Use Attention for Slot Filling
4. Leverage Intent Information for Slot Filling
5. Conditional Random Field (CRF) for Slot Tagging

Training

1. Fine-Tune the Pre-Trained BERT Model
2. Use Class Weights for Imbalanced Datasets
3. Apply a Learning Rate Scheduler
4. Adjust Loss Weights
5. Add Weight Decay
6. Gradient Clipping
7. DataLoader Improvements
8. Regularization with Dropout

Data

1. Handle Data Imbalance
2. Introduce Domain-Specific Tokenization
3. Improve Slot Label Alignment
4. Expand Dataset with Augmentation
5. Use Dynamic Padding
6. Include Contextual Features
7. Implement Dataset Splitting with Stratification
8. Pre-Encode Ontology Information
9. Improve BIO Tagging

Other extensions

- 1. Negation query understanding (Excluding Features);
- 2.
- ...
- And more!

And more!

```
{
  "id": "ec010",
  "text": "Avoid French food, please.",
  "intent": "addFilter",
  "slots": {
    "excludeCuisine": "French"
  }
},
{
  "id": "ec123456-7890-1234-5678-abcdef123456",
  "text": "No Italian cuisine, please.",
  "intent": "addFilter",
  "slots": {
    "excludeCuisine": "Italian"
  }
},
```

Hands-on: develop a NLU model
with

<https://socialrobotics.atlassian.net/wiki/spaces/PCA21/pages/3117779631/Your+Intent+and+Slot+Classifier>

on confluence!



Overview

- SLU and its Connection
- Language Modeling with Transformers
- Introduction of ASR with WHISPER
- Your NLU Model with BERT
- **Demo & Test Cases**
- Summary

A quick look at the running demo

```
redis-server conf/redis/redis.conf &  
run-whisper &  
run-nlu &
```

```
Successfully installed social-interaction-cloud-2.0.23  
(PCA25) j1shuanpei@VU-WMP-KP9RYQNT8 social-interaction-cloud % redis-server conf/redis/redis.conf &  
run-whisper &  
run-nlu  
(1) 27012  
(2) 27013  
27012:C 09 Jan 2025 05:36:25.722 * o000o00o000o Redis is starting o000o00o000o  
27012:C 09 Jan 2025 05:36:25.722 * Redis version=7.2.6, bits=64, commit=08080808, modified=0, pid=27012, just started  
27012:C 09 Jan 2025 05:36:25.722 * Configuration loaded  
27012:M 09 Jan 2025 05:36:25.722 * monotonic clock: POSIX clock_gettime  
-----  
Redis 7.2.6 (08080808/0) 64 bit  
Running in standalone mode  
Port: 6379  
PID: 27012  
  
https://redis.io  
-----  
27012:M 09 Jan 2025 05:36:25.725 # WARNING: The TCP backlog setting of 511 cannot be enforced because kern.ipc.somaxconn is set to the lower value of 128.  
27012:M 09 Jan 2025 05:36:25.725 # Warning: Could not create server TCP listening socket *:6379: bind: Address already in use  
27012:M 09 Jan 2025 05:36:25.725 # Failed listening on port 6379 (tcp), aborting.  
(1) ~ exit 1 redis-server conf/redis/redis.conf  
[SICComponentManager 192.168.2.12]: INFO: Manager on device 192.168.2.12 starting  
[SICComponentManager 192.168.2.12]: INFO: Started component manager on ip "192.168.2.12" with components:  
[SICComponentManager 192.168.2.12]: INFO: - WhisperComponent  
[SICComponentManager 192.168.2.12]: INFO: Manager on device 192.168.2.12 starting  
[SICComponentManager 192.168.2.12]: INFO: Started component manager on ip "192.168.2.12" with components:  
[SICComponentManager 192.168.2.12]: INFO: - NLUComponent  
SICComponentManager ignored request WhisperComponent  
SICComponentManager handling request WhisperComponent  
[WhisperComponent 192.168.2.12]: INFO: Started component WhisperComponent  
[WhisperComponent 192.168.2.12]: INFO: Inferred sample rate: 44100 and chunk size: 2730
```

```
python sic_applications/demo_asr_nlu.py
```

```
[NLUComponent 145.108.197.79]: INFO: Started component NLUComponent  
Initiated NLU component!  
-----  
..... Talk now Round 0  
[NLUComponent 145.108.197.79]: INFO: Running inference on example text: Can you recommend a recipe?  
[NLUComponent 145.108.197.79]: INFO: Inference result: requestRecommendation, 0.8632935881614685, {}, 0.85596598042114258  
Transcript: Can you recommend a recipe?  
Intent: requestRecommendation 0.8632935881614685  
Slots:  
{}  
-----  
..... Talk now Round 1  
[NLUComponent 145.108.197.79]: INFO: Running inference on example text: Add garlic and chicken sauce to the recipe filter.  
[NLUComponent 145.108.197.79]: INFO: Inference result: addFilter, 0.9366124272346497, {'ingredient': 'chicken sauce'}, 0.84708883756713867  
Transcript: Add garlic and chicken sauce to the recipe filter.  
Intent: addFilter 0.9366124272346497  
Slots:  
{'ingredient': 'chicken sauce'}  
-----  
..... Talk now Round 2  
[NLUComponent 145.108.197.79]: INFO: Running inference on example text: I'd like a mule that is fast to prepare.  
[NLUComponent 145.108.197.79]: INFO: Inference result: addFilter, 0.8812377452850342, {'duration': 'prepare.'}, 0.86378698348999823  
Transcript: I'd like a mule that is fast to prepare.  
Intent: addFilter 0.8812377452850342  
Slots:  
{'duration': 'prepare.'}  
-----  
..... Talk now Round 3  
[NLUComponent 145.108.197.79]: INFO: Running inference on example text: How to cook an Italian pasta?  
[NLUComponent 145.108.197.79]: INFO: Inference result: recipeRequest, 0.9194018840789795, {'recipe': 'italian pasta?'}, 0.84758310317993164  
Transcript: How to cook an Italian pasta?  
Intent: recipeRequest 0.9194018840789795  
Slots:  
{'recipe': 'italian pasta?'}  
-----  
..... Talk now Round 4  
[NLUComponent 145.108.197.79]: INFO: Running inference on example text: Find me a Japanese recipe for launch.  
[NLUComponent 145.108.197.79]: INFO: Inference result: addFilter, 0.9576860771942139, {'cuisine': 'japanese'}, 0.84436516761779785  
Transcript: Find me a Japanese recipe for launch.  
Intent: addFilter 0.9576860771942139  
Slots:  
{'cuisine': 'japanese'}  
-----
```

- Can you recommend a recipe? (requestRecommendation)
- Add garlic and chicken thighs to the recipe filter (ingredient=garlic and chicken thighs)
- I'd like a meal that's fast to prepare (duration=fast)
- How to cook an italian pasta (recipe=italian pasta)
- Find me a Japanese recipe for lunch (cuisine=japanese, mealType=lunch)

Test cases

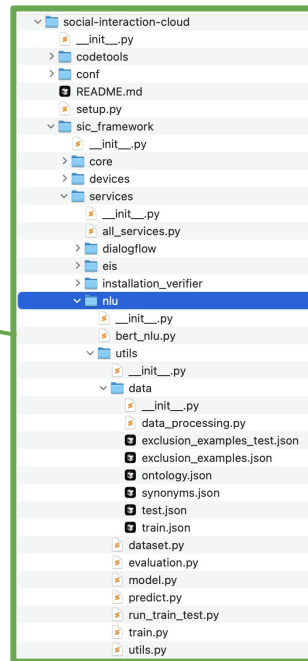
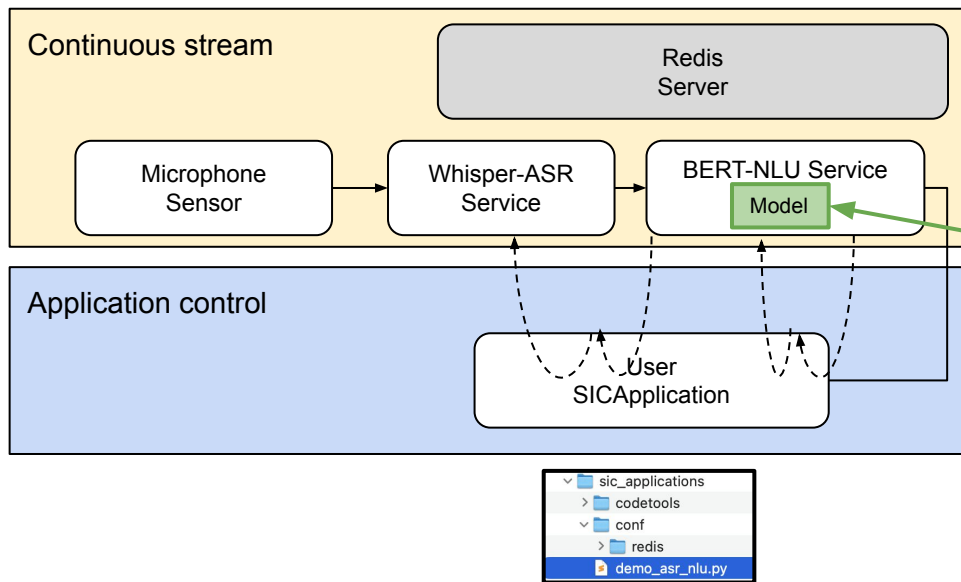


Try it out!

Your ASR+NLU pipeline deployed on SIC

- Whisper-ASR and BERT-NLU on SIC-v2

**Develop and deploy
your NLU model here!**



More SIC components at

<https://socialrobotics.atlassian.net/wiki/spaces/CBSR/pages/2178973697/Available+services>

Information from ASR+NLU pipeline to MARBEL

Data Item	Description	Usage in EISComponent
Intent Name	The detected goal of the user's input (e.g., "GetWeather").	Used to trigger actions or send percepts to MARBEL agents.
Entities (Slots)	Key-value pairs extracted from input (e.g., city=London).	Provide specific information for actions or percepts.
Transcript	The raw text entered by the user (spoken or typed).	Sent to the webserver for real-time display.
Confidence Scores	Dialogflow's confidence in the identified intent or slots.	Helps determine whether to trust the NLU result or ask the user for clarification.
Source Type	Indicates whether the request originated from speech or text input.	Adds context to actions or percepts sent to MARBEL.
Recognition Results	Real-time transcription feedback during speech recognition.	Displays transcriptions in real-time on the webserver for UI feedback.

Connecting Your Entire Pipeline

Instructions for Connecting and Running the Pipeline

 YOU CANNOT RUN THIS PART UNTIL YOU ARE DONE WITH YOUR ENTIRE INTENT AND SLOT CLASSIFIER.

Setting Up the New Pipeline Connection

To connect your pipeline, follow these updated instructions carefully. Ensure you meet the prerequisites and run the commands in the correct sequence.

Prerequisites:

1. Activate PCA26:

Ensure that PCA26 is activated in your environment.

2. Model Checkpoints:

Confirm you have a trained model checkpoint file (`model_checkpoint.pt`) saved in the `nlu.utils.checkpoints` folder. All the members in your group need to have the same checkpoints file in the folder.

Steps to Run the Pipeline:

Run each of the following commands in **separate terminals** **after running your redis server**:

 First do `pip install -e .` in both `social-interaction-cloud` and `sic_applications` directories.

 Please keep in mind that you always need to be in the conda environment `PCA26`, which we created during installation, in order to run your agent.

Repository Overview

 Find the Intent and Slot Classifier files under `/pca-agent-2025/social-interaction-cloud/sic_framework/services/nlu/utils`

```
1 The repository is structured as follows:
2 utils/
3 |─ checkpoints/           # Directory for saving trained model checkpoints
4 |─ data/                 # Directory containing data files (ontology, train/test datasets)
5 |   │─ ontology.json     # Ontology file containing intents, slots, and synonyms
6 |   │─ train.json        # Training dataset
7 |   │─ test.json         # Test dataset
8 |   │─ synonyms.json     # Synonyms for slot normalization
9 |─ data_processing.py    # Utilities for additional data preprocessing (if needed)
10 |─ dataset.py            # Dataset preparation and preprocessing module
11 |─ evaluation.py         # Model evaluation and metrics generation
12 |─ run_train_test.py     # Main script to run training, evaluation, and inference
13 |─ model.py              # Defines the BERT-based model architecture
14 |─ predict.py            # Inference module for predicting intents and slots
15 |─ requirements.txt      # Python dependencies for the project
16 |─ train.py              # Training module for the intent-slot classifier
17 |─ utils.py              # Helper functions for argument parsing, slot normalization, and synonym resolution
18
```

Connecting Your Entire Pipeline

Launch the Redis Server

1. Open the project in an IDE (e.g., VS Code, PyCharm, etc.), or a terminal.
2. Go to your cloned repository.
3. Navigate to the `sic_applications` directory:

```
1 cd sic_applications
```

4. Start the Redis server using the provided configuration file:

```
1 redis-server conf/redis/redis.conf
```

For Windows go into `sic_applications/conf/` and you can run `redis-server.exe`

If you encounter this error `Could not create server TCP listening socket *:6379: bind: Address already in use` read through the following collapsable section.

> Error Problem Solving

Connecting Your Entire Pipeline

1. Run ASR (Automatic Speech Recognition) - Google STT or Whisper:

If you have activated the Google STT API, use the following command:

```
1 run-google-stt
```

Otherwise, start the Whisper component with this command:

```
1 run-whisper
```

Note: The first time you use Whisper, the transcription may take some time due to downloading the model. Be patient during this process. If it still takes a long time after the first time read the top of the `eiscomponent.py` file.

 It is entirely up to you whether you will use Google STT or Whisper with your Intent & Slot Classifier. Feel free to choose the one that works best for you.

2. Run NLU:

Start the natural language understanding (NLU) module:

```
1 run-nlu
```

3. Run Webserver:

Launch the webserver for pipeline integration:

```
1 run-webserver
```

4. Start Framework:

Initialize the framework:

```
1 start-framework
```

5. Run EIS:

Start the External Interface Service (EIS):

```
1 run-eis
```

Note keyword arguments:

There are several keyword arguments you can pass to the `run-eis` script that will change what components the pipeline uses. You can find these in the main function of `run-eis`, but they are also in the table below.

For example, to use your intent and slot classifier, you will pass in `--use-nlu` as an argument like so (don't run this now, just take note):

```
1 run-eis --use-nlu
```

To use Whisper instead of Google STT, you would run the following command:

```
1 run-eis --use-whisper
```

<code>run-eis</code> Argument	Meaning
<code>--use-nlu</code>	Will use your NLU component rather than Dialogflow
<code>--use-whisper</code>	Will use Whisper rather than Google STT
<code>--no-espeak</code>	Will use Google TTS rather than <code>espeak</code>

6. Launch Localhost:

Open your browser and navigate to the localhost start page (<http://localhost:8080/start.html>).

7. Debug MARBEL Agent in Eclipse:

Start the MARBEL agent within Eclipse. Ensure the agent connects successfully to the pipeline.

Demo

Try it out!

- [Basic Agent with ASR+NLU](#)

Check out the how to run
ASR+NLU pipeline demo
on confluence!

Overview

- SLU and its Connection
- Language Modeling with Transformers
- Introduction of ASR with WHISPER
- Your NLU Model with BERT
- Demo & Test Cases
- Summary

Summary

This week: *Check out the [course schedule!](#)*

- Finish the project installation process
- Start developing your conversational agent
- Work on project assignment and Commit to GitHub
- Prepare for the Exam & Midway Check-In Presentation

Thank you for your attention!

- Q & A
- Your feedback for how we can do better

<https://docs.google.com/document/d/1GCIJL6l4RO7kpYEA6yD530g1WCh11hRn-W7G8uMTKA/edit?usp=sharing>